

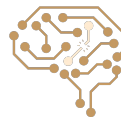
Neural Networks Part I

Arash Marioryad

17 Khordad 1405

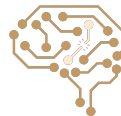
CE 417-Artificial Intelligence, Sharif Univ. of Tech.

Most Slides have been adopted from Berkeley CS188 & SBU CI course Dr. Malek & Sharif UT CE417



Components of Supervised Learning

- Input space: $\mathcal{X}$
- Output space: $\mathcal{Y}$
- Unknown target function: $f: \mathcal{X} \rightarrow \mathcal{Y}$
- Training data: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
- Pick a formula $g: \mathcal{X} \rightarrow \mathcal{Y}$ that approximates the target function f
- Hypothesis Set $\mathcal{H}$: a set (class) of functions that g comes from



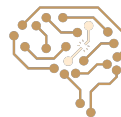
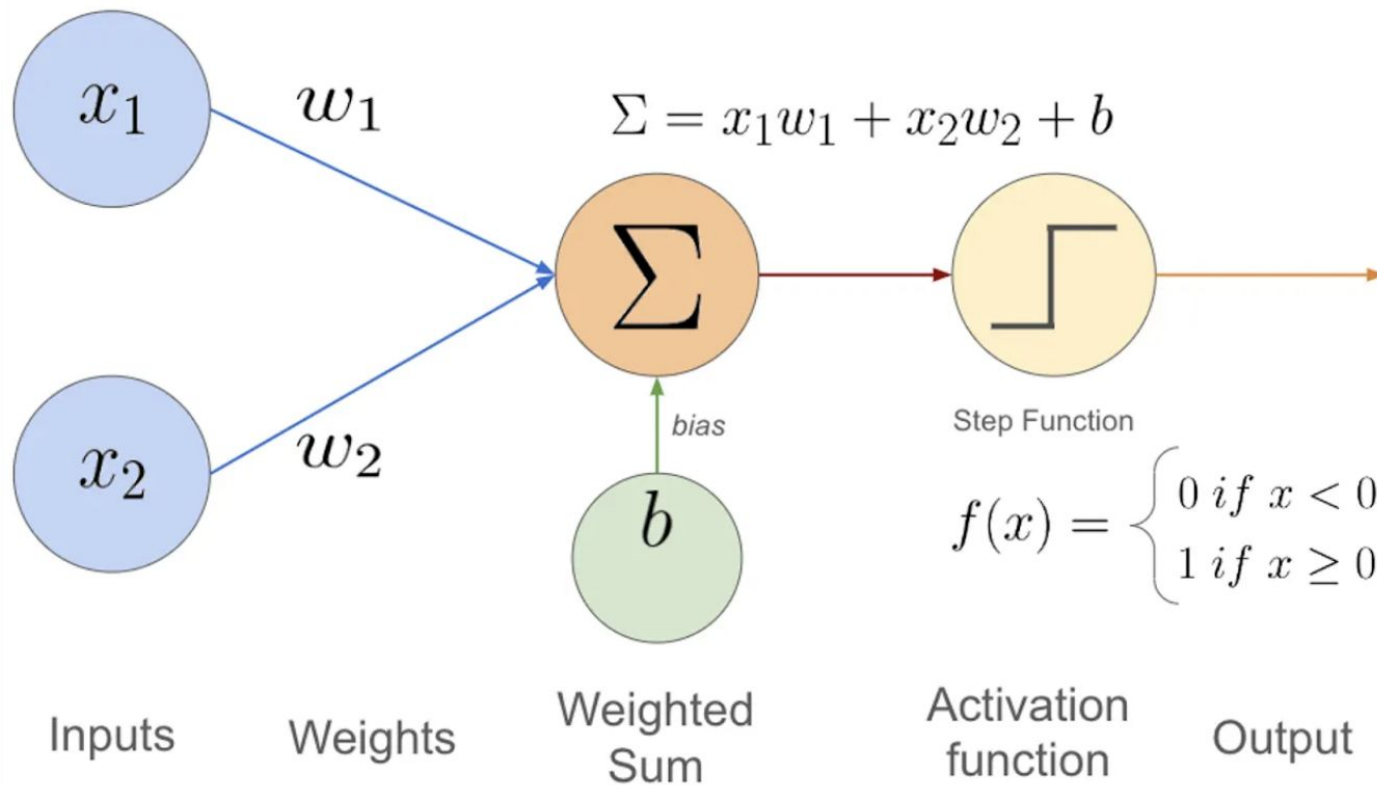
g Functions So Far

- Naive Bayes
- Decision Tree
- Linear Regression
- Perceptron
- Logistic Regression

- Today: Neural Networks



Perceptron



Learning Algorithm of Perceptron

- ▶ Given N training instances $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})$
 - ▶ $y^{(n)} = +1$ or -1

- Initialize $\mathbf{w}$
- Cycle through the training instance
- While more classification errors

- For $i = 1 \dots N_{train}$
 - $\hat{y}^{(i)} = \text{sign}(\mathbf{w}^T \mathbf{x}^{(i)})$
 - If $\hat{y}^{(i)} \neq y^{(i)}$
 - $\mathbf{w} = \mathbf{w} + y^{(i)} \mathbf{x}^{(i)}$

If instance misclassified:

– If instance is positive class

$$\mathbf{w} = \mathbf{w} + \mathbf{x}$$

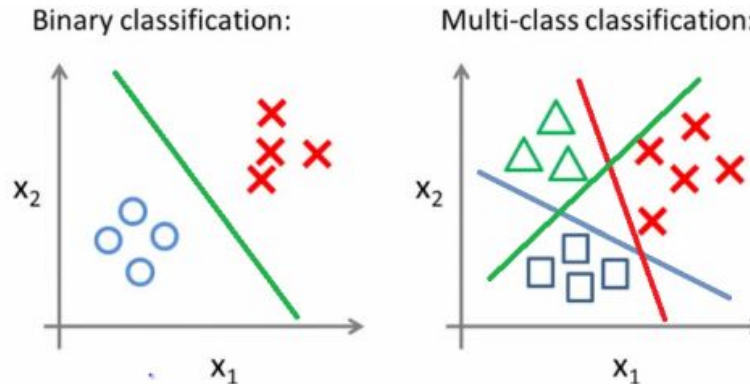
– If instance is negative class

$$\mathbf{w} = \mathbf{w} - \mathbf{x}$$



Multi-class Perceptron

- In multiple classes classification, we have a weight vector for each class
- Score of a class y : $\langle W^{(y)}, X \rangle$
- Prediction class: highest score $y = \arg \max_y \langle W^{(y)}, X \rangle$



Multi-class Perceptron: Learning

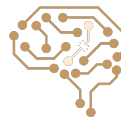
- Start with all weights = 0
- Pick up training examples one by one
- Predict the class of sample:

$$y = \arg \max_y \langle W^{(y)}, X \rangle$$

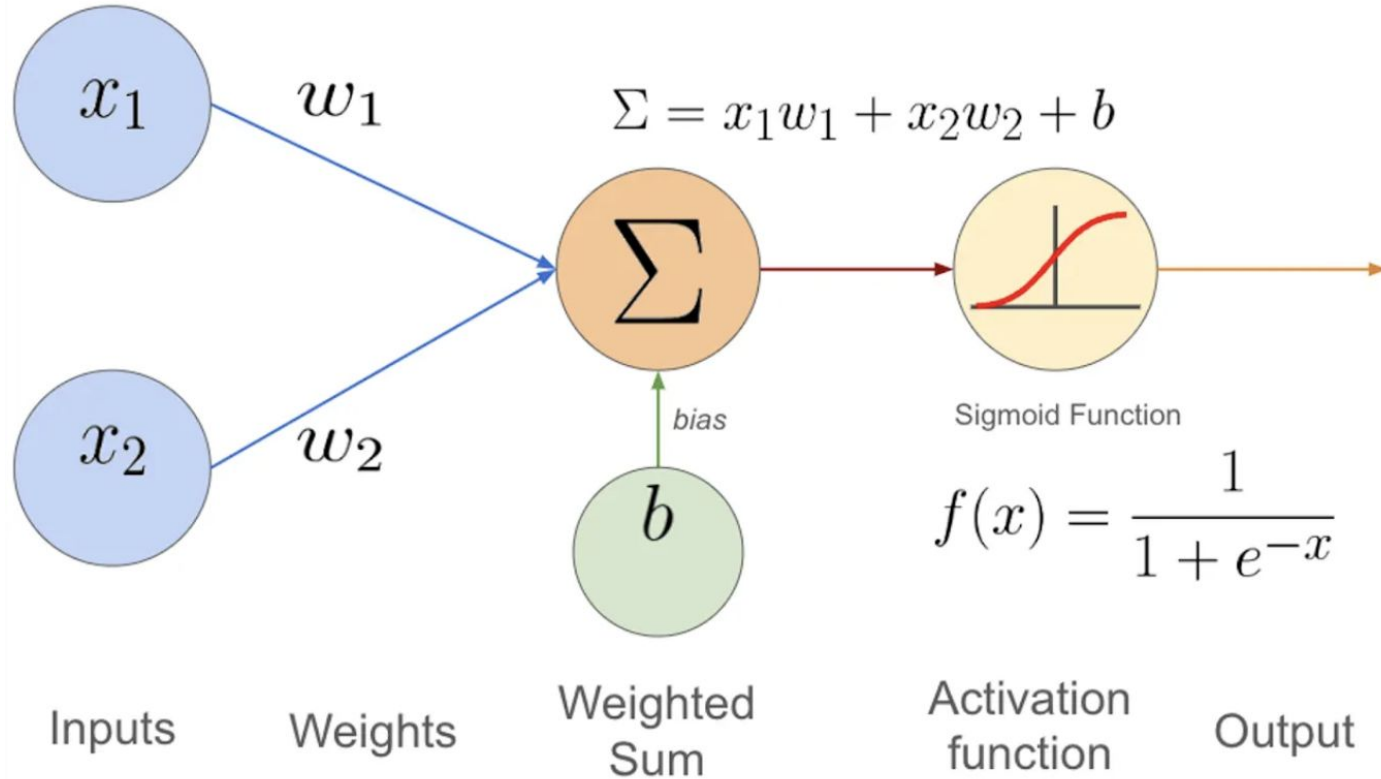
- If correct, no change!
- If wrong: lower score of wrong answer, raise score of right answer

$$W^{(y)} = W^{(y)} - X$$

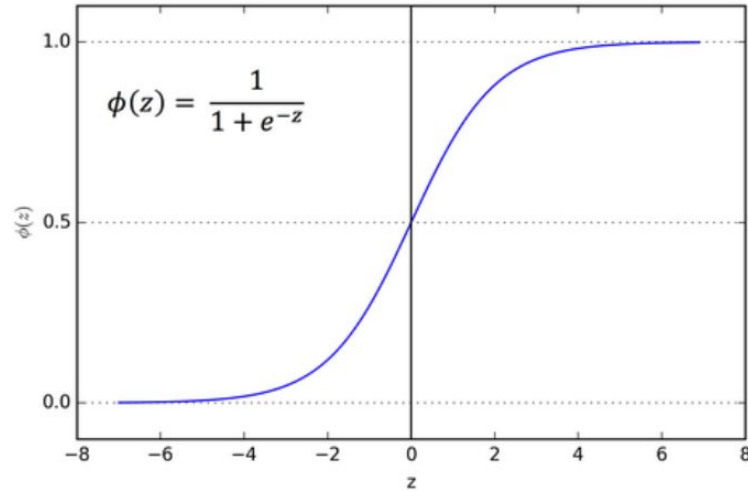
$$W^{(y^*)} = W^{(y^*)} + X$$



Logistic Regression



Sigmoid Activation Function



Binary Cross Entropy (BCE) Cost Function

$$J(\mathbf{w}) = \sum_{i=1}^n -y^{(i)} \log(\sigma(\mathbf{w}^T \mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}))$$

$J(\mathbf{w})$ is convex w.r.t. parameters.

Since $y = 1$ or $y = 0$

$$\Rightarrow \text{Loss}(y, \sigma(\mathbf{x}; \mathbf{w})) = \begin{cases} -\log(\sigma(\mathbf{x}; \mathbf{w})) & \text{if } y = 1 \\ -\log(1 - \sigma(\mathbf{x}; \mathbf{w})) & \text{if } y = 0 \end{cases}$$



Logistic Regression: Learning

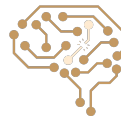
- The activation function is Sigmoid, which is differentiable
- The cost function: $\sum_{i=1}^n -y^{(i)} \log(\sigma(\mathbf{w}^T \mathbf{x}^{(i)})) - (1 - y^{(i)}) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}))$
- Gradient Descent:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{i=1}^n (\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)}$$

- It is similar to gradient update for the SSE loss in Linear Regression!

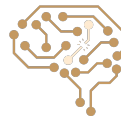
$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$



Multiclass Logistic Regression

- We have a weight vector for each class
- Score of a class y : $z^{(y)} = \langle W^{(y)}, X \rangle$
- We can not use sigmoid anymore
- Softmax: an extension of sigmoid for multi-class scenarios

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

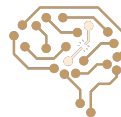


Multiclass Logistic Regression (cont.)

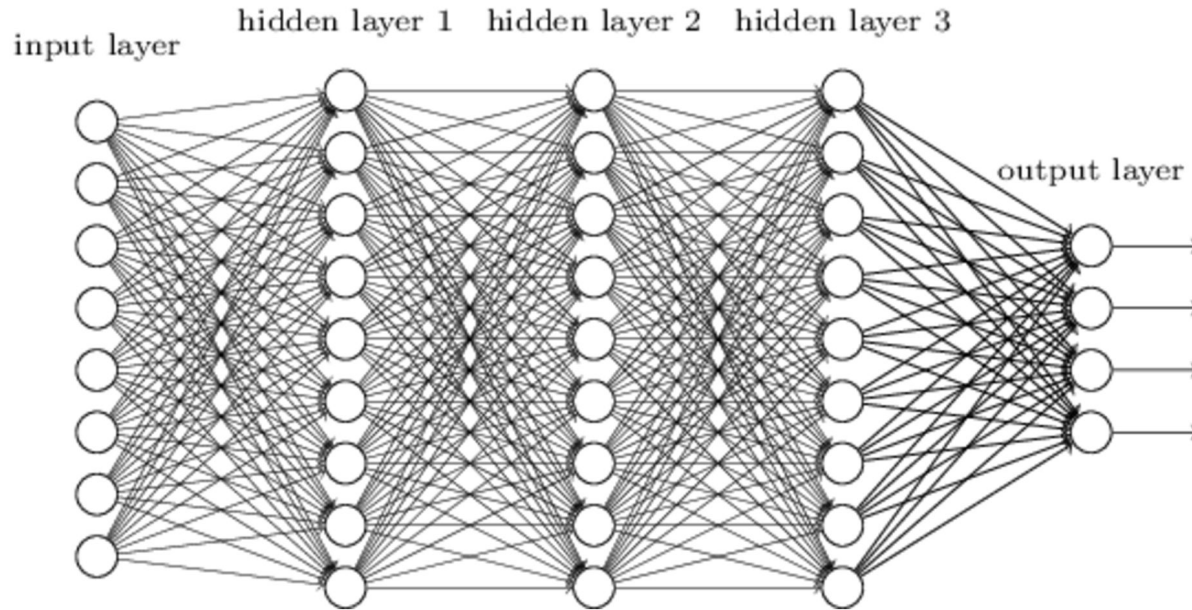
- Loss Function (for one sample):

$$L_{CE}(\hat{y}, y) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- Note that for a specific value of k , y_k is 1, and for all others it is 0

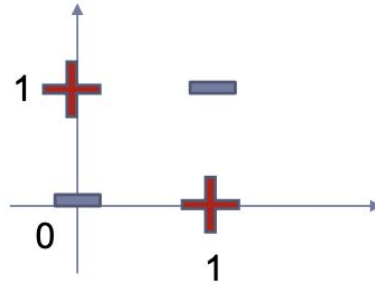


Today: Neural Networks



The XOR function

- One of the main weakness of linear models is that they are linear
- Note that a perceptron cannot learn the XOR function

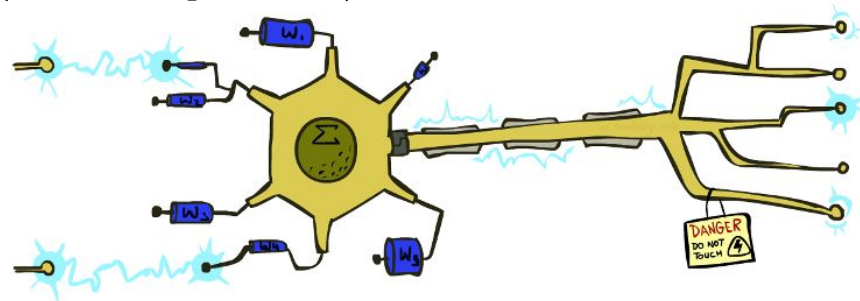


Reminder: Linear Classifiers (Perceptron)

Inputs are **feature values**

Each feature has a **weight**

Sum is the **activation**

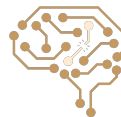
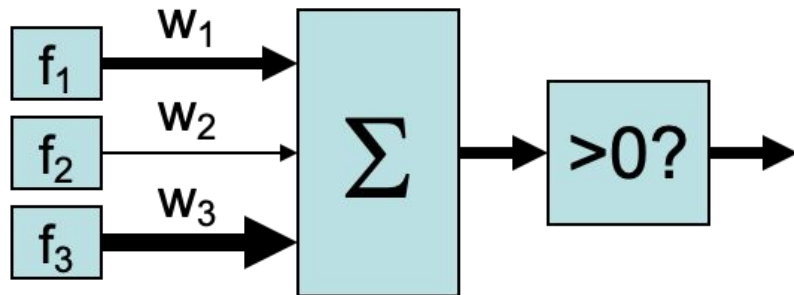


$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

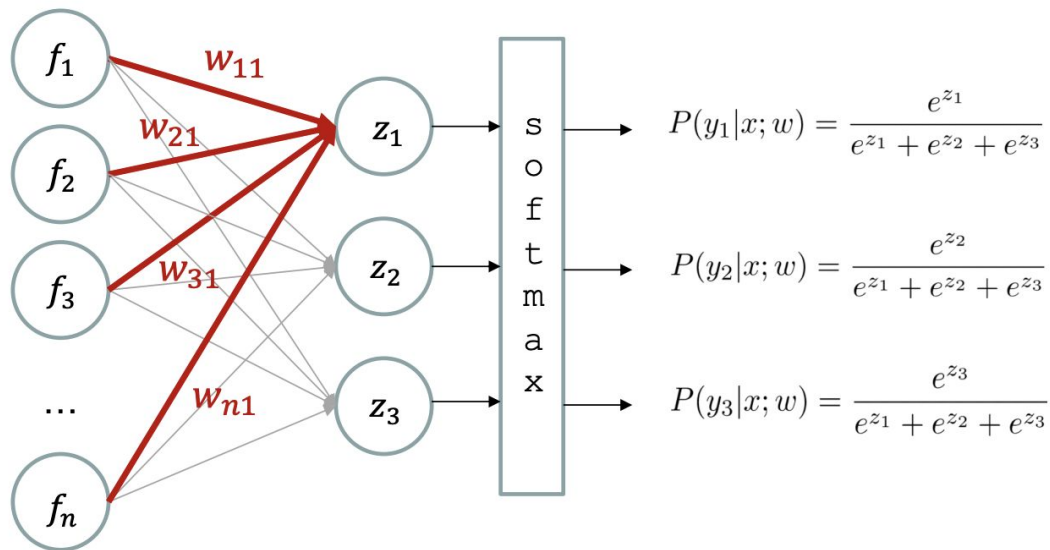
If the activation is:

Positive, output +1

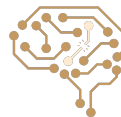
Negative, output -1



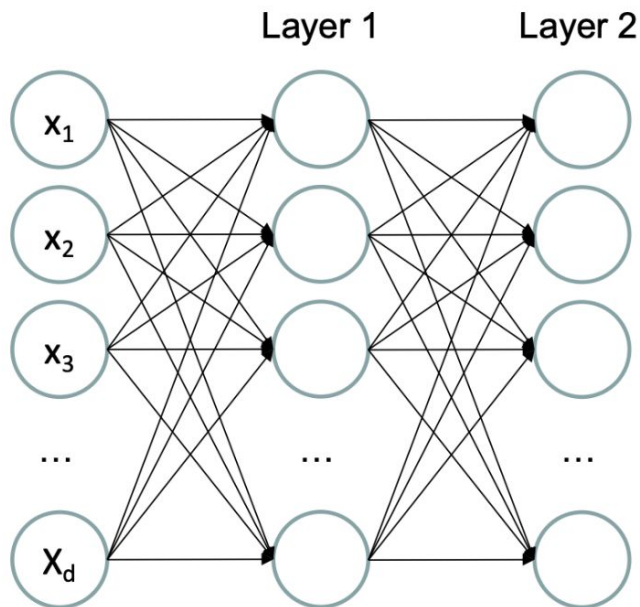
Logistic Regression for 3-way classification



$$\begin{aligned} z_i &= w_i \cdot f \\ &= \sum_j w_{ji} \cdot f_j \end{aligned}$$

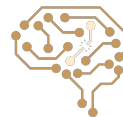
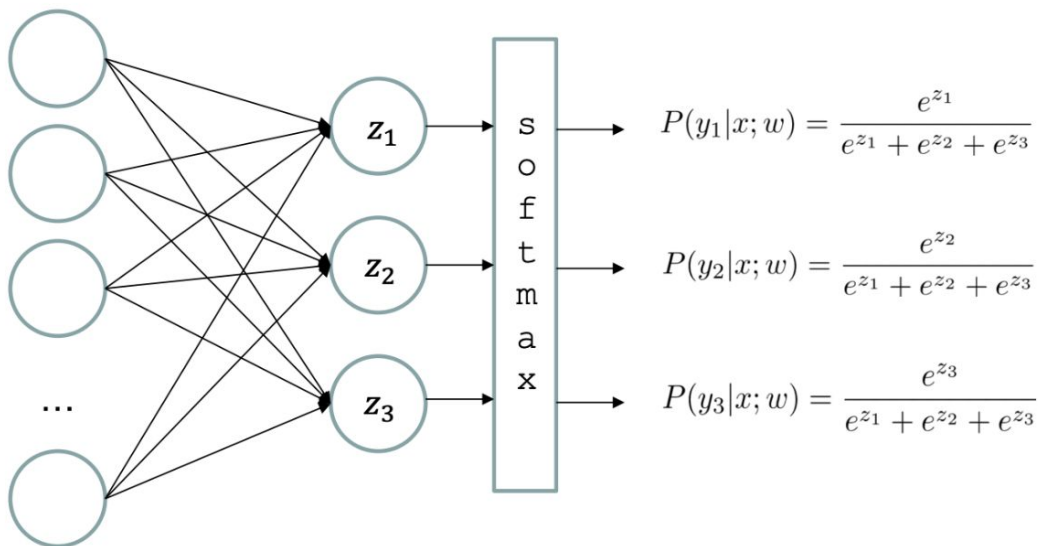


Deep Neural Network for 3-way classification



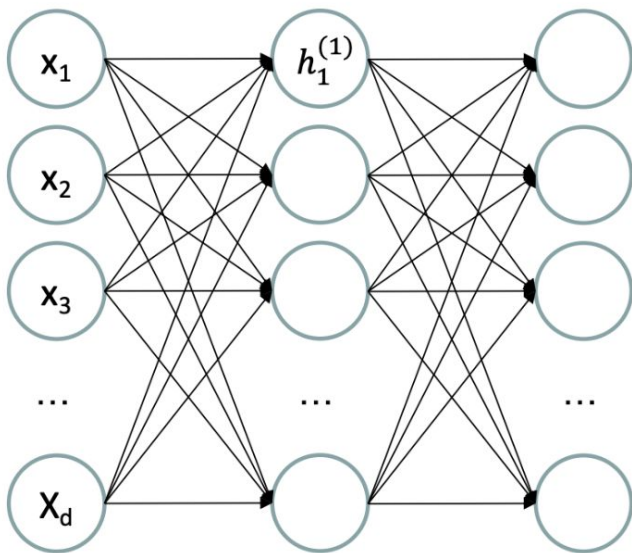
...

Layer L

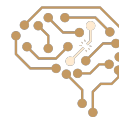
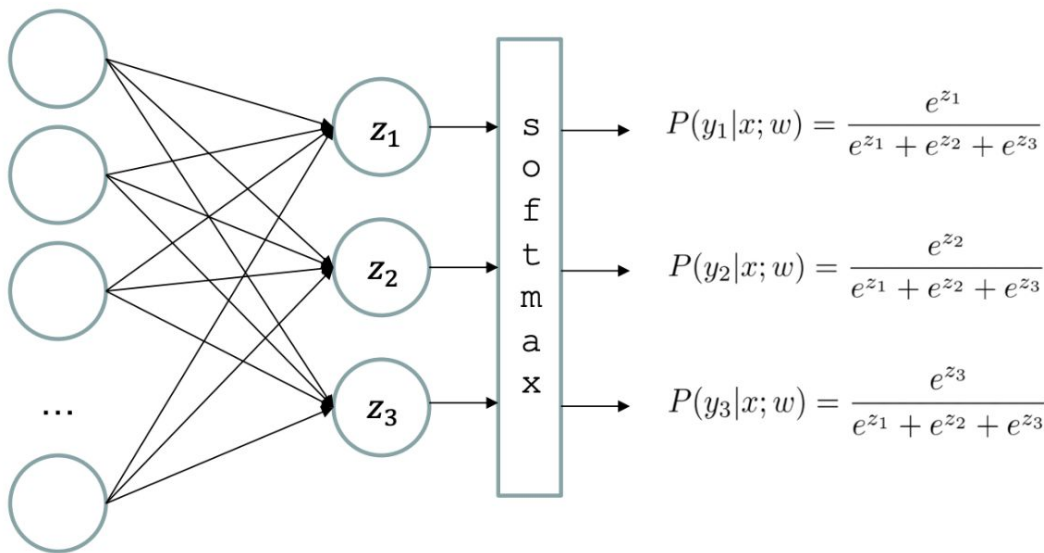


Deep Neural Network for 3-way classification

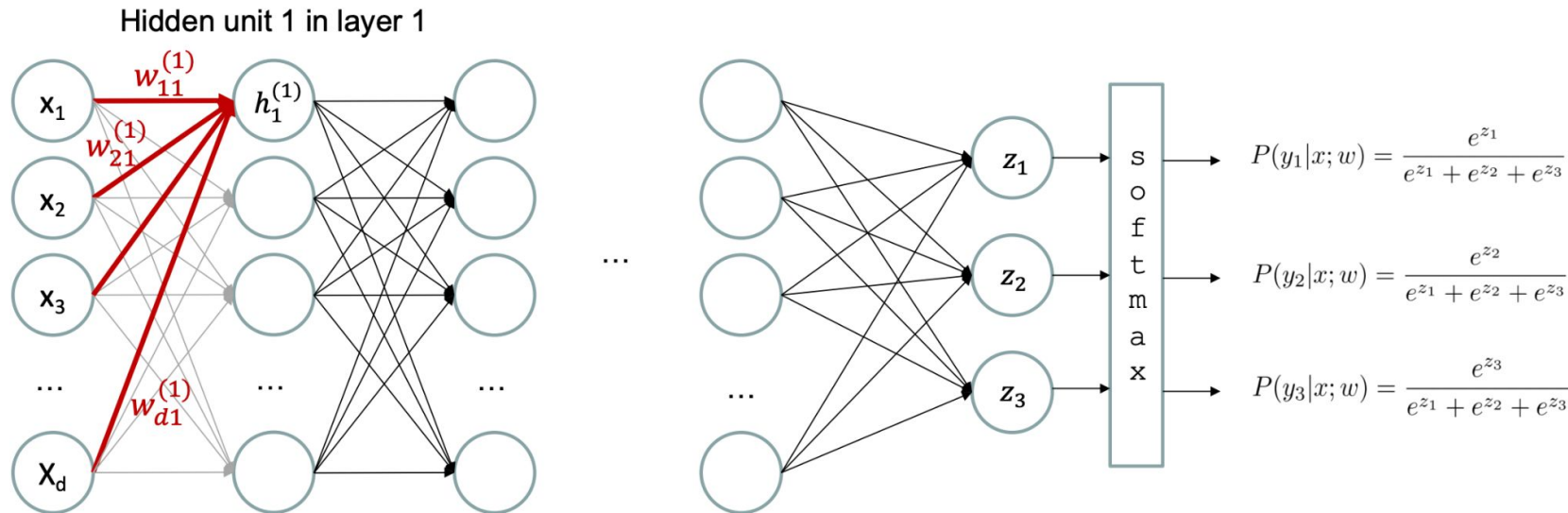
Hidden unit 1 in layer 1



...

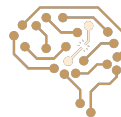


Deep Neural Network for 3-way classification



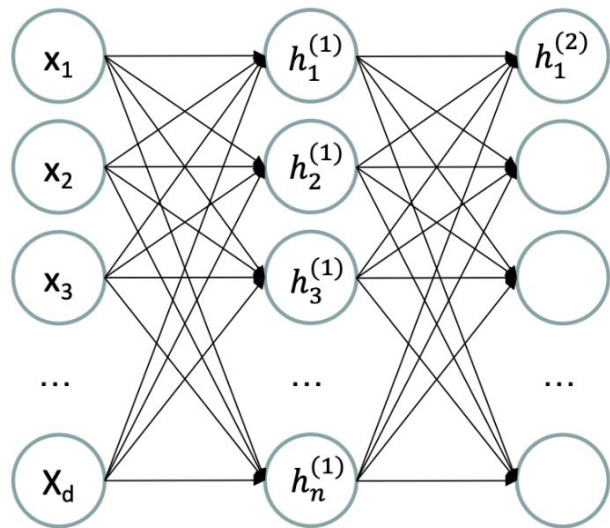
$$h_1^{(1)} = \phi \left(w_1^{(1)} \cdot x \right) = \phi \left(\sum_j w_{ji}^{(1)} \cdot x_j \right)$$

ϕ = activation function

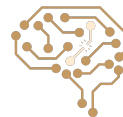
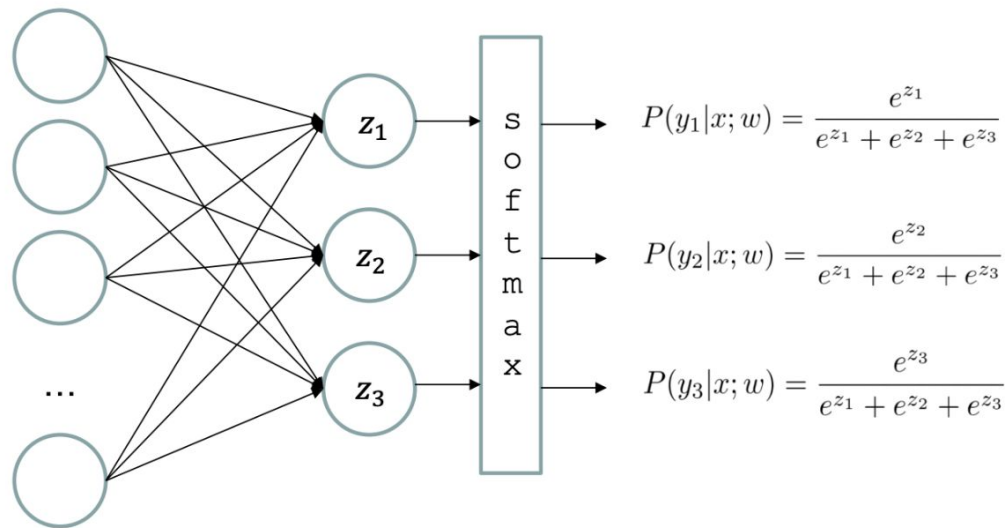


Deep Neural Network for 3-way classification

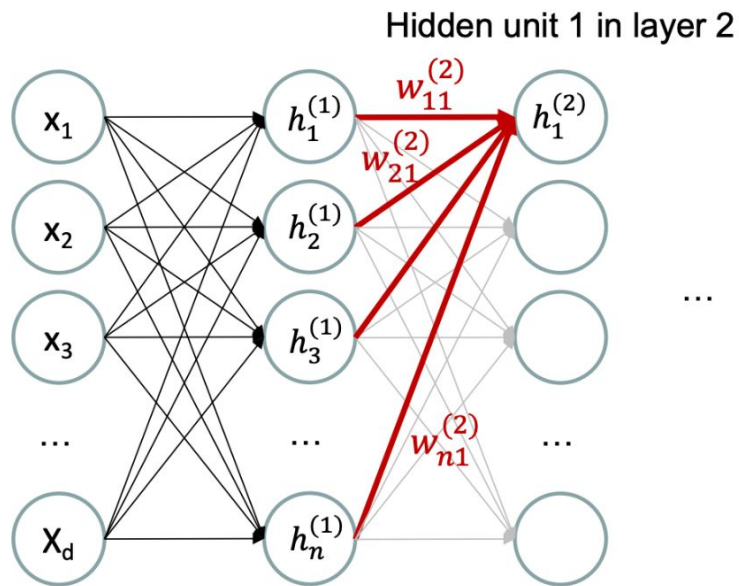
Hidden unit 1 in layer 2



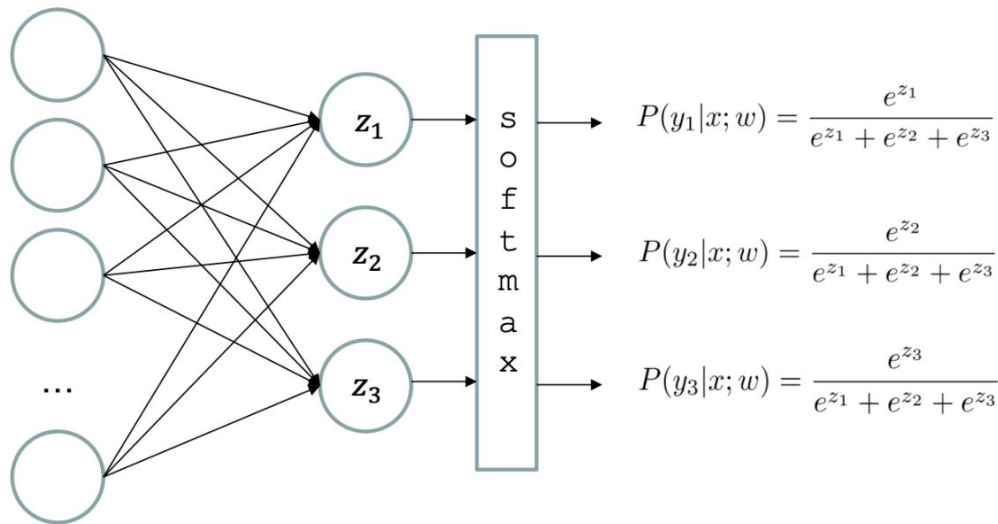
...



Deep Neural Network for 3-way classification



...

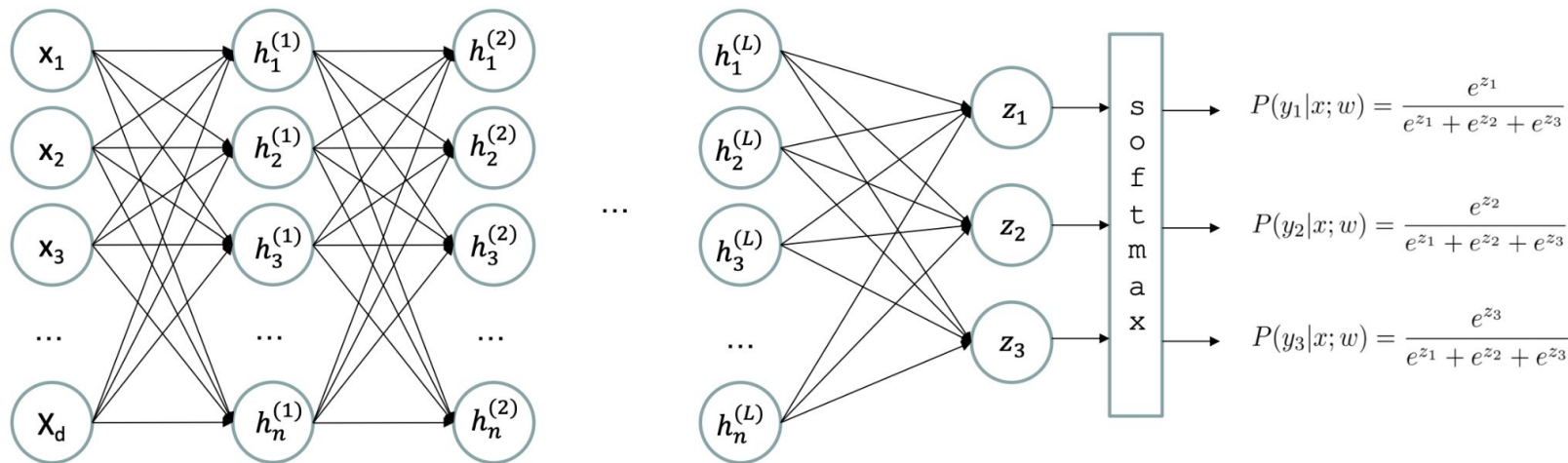


$$h_1^{(2)} = \phi \left(w_1^{(2)} \cdot h^{(1)} \right) = \phi \left(\sum_j w_{ji}^{(2)} \cdot h_j^{(1)} \right)$$

ϕ = activation function



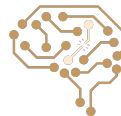
Deep Neural Network for 3-way classification



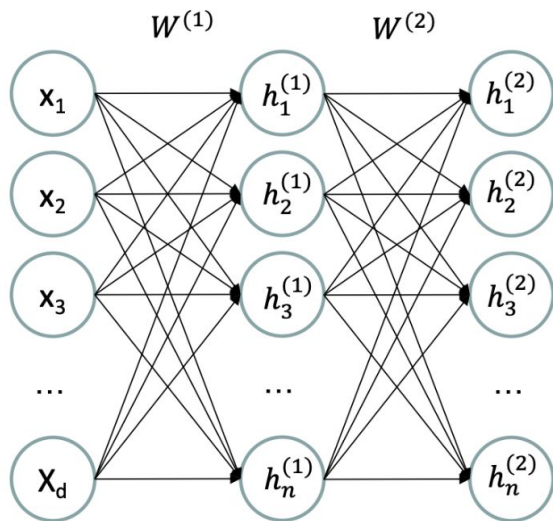
$$h_i^{(l)} = \phi\left(\sum_j w_{ji}^{(l)} \cdot h_j^{(l-1)}\right)$$

ϕ = activation function

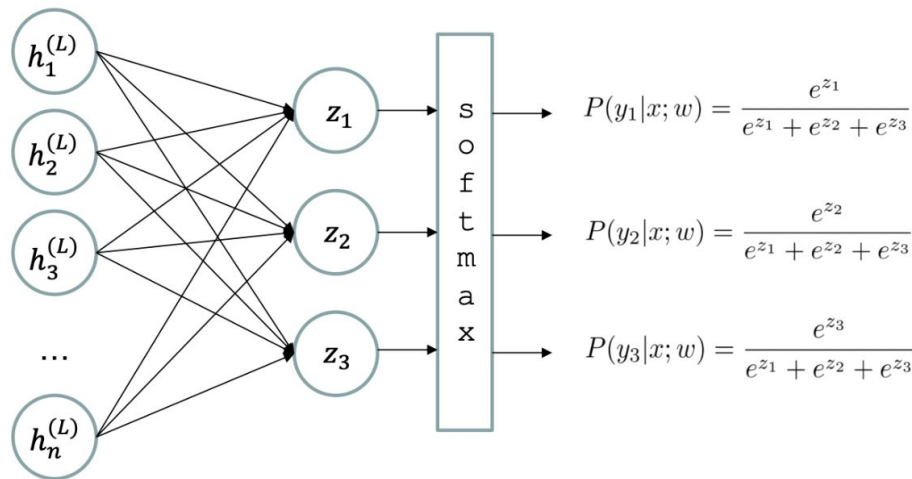
- Neural network with L layers
- $h^{(l)}$: activations at layer l
- $w^{(l)}$: weights taking activations from layer l-1 to layer l



Deep Neural Network for 3-way classification

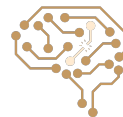


...

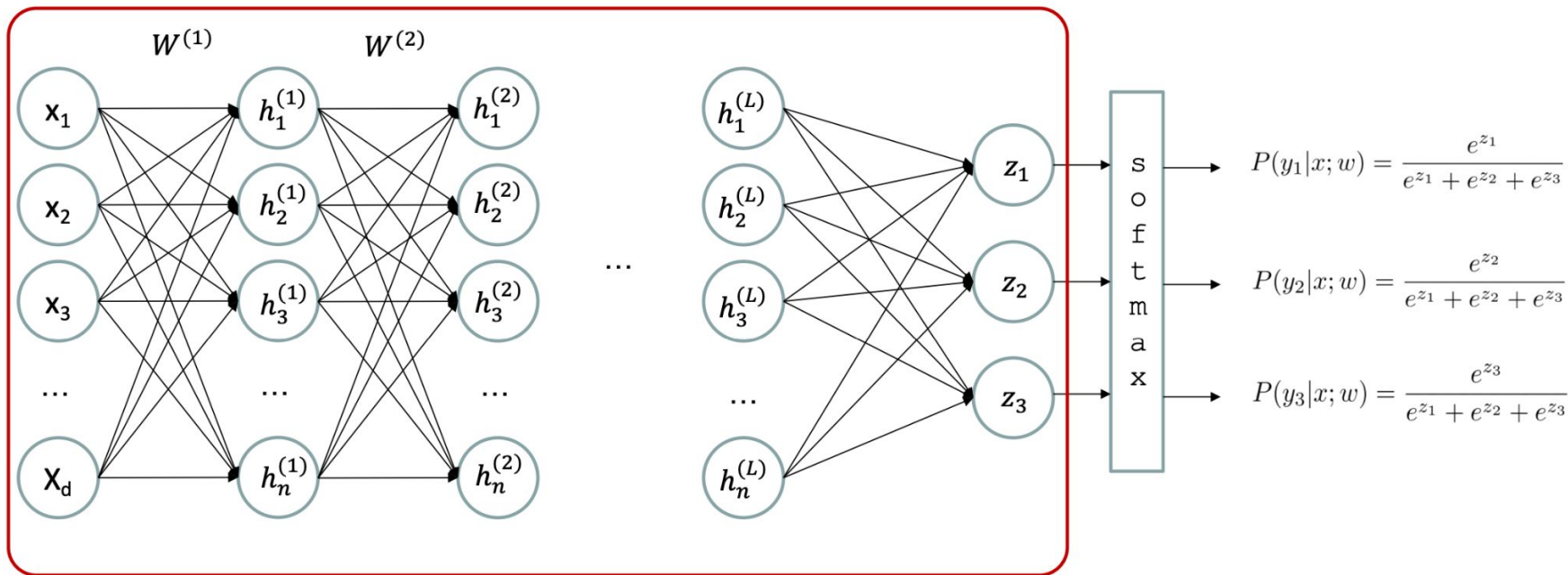


$$h^{(l)} = \phi(h^{(l-1)} \times W^{(l)})$$

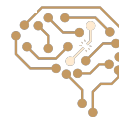
ϕ = activation function



Deep Neural Network for 3-way classification



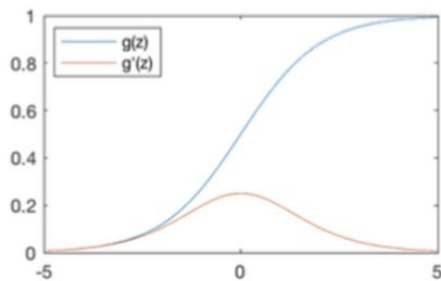
- Sometimes also called *Multi-Layer Perceptron (MLP)* or *Feed-Forward Network (FFN)*



Non Linearity is Important

- Common Activation Functions

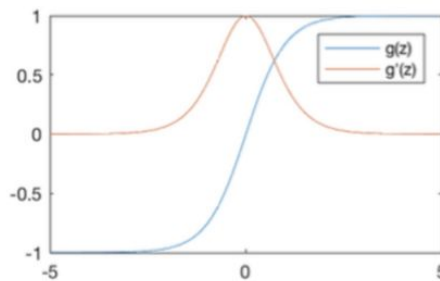
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

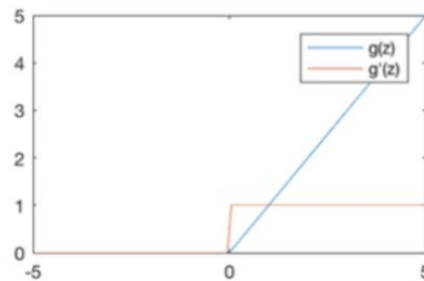
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

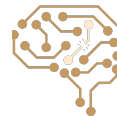
$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$



Fun Neural Net Demo Site

Demo site: <http://playground.tensorflow.org/>



How Many Hidden Layers?

- More hidden layers means a more complex model
- More complex model means the risk of overfitting



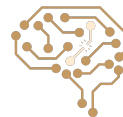
Deep Neural Network Learning

- Given: the architecture of the network
- Training data: A set of input-output pairs

$$(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(N)}, \mathbf{y}^{(N)})$$

- We want to find the function g (model) on the input to get the output
- We consider a neural network as a parametric function $g(\mathbf{x}; \mathbf{W})$
- We need a **loss function** to show how penalizes the obtained output $g(\mathbf{x}; \mathbf{W})$ when the desired output is $\mathbf{y}$

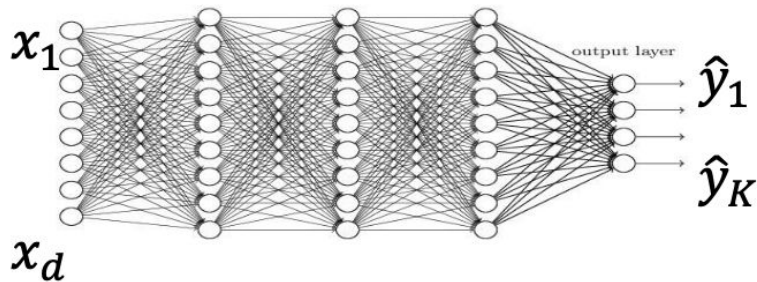
$$\frac{1}{N} \sum_{n=1}^N \text{loss}(g(\mathbf{x}^{(n)}; \mathbf{W}), \mathbf{y}^{(n)})$$



Choosing Cost Function: Examples

- > Regression problem (multiple outputs)

SSE Loss



$$E = \sum_{n=1}^N E_n$$

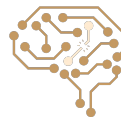
$$E_n = \sum_{k=1}^K \left(\hat{y}_k^{(n)} - y_k^{(n)} \right)^2$$

- > Classification problem

Cross Entropy Loss

$$loss_n = \sum_{k=1}^K -y_k^{(n)} \log \hat{y}_k^{(n)}$$

Output is found by a softmax layer $\hat{y}_k = \frac{e^{z_k}}{\sum_{k=1}^K e^{z_k}}$



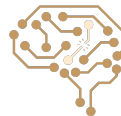
Deep Neural Network Training

- Do the maximum likelihood estimation is a good Idea ...
- Find a set of parameters which results in the best performance in training data
- In general, cannot always take derivative and set to 0
- Use numerical optimization!
- **GRADIENT DESCENT!**
- Hill climbing in continuous space



The Main Issue of Using Gradient Descent for NNs

- The loss function (L) itself is a function of model g
- The model g itself is a complex function of weights (parameters) $W^{(1)}$, $W^{(2)}$, ..., $W^{(L)}$
- We want the gradient of the loss function w.r.t to the parameters



Review: Derivatives and Gradients

What is the derivative of the function $g(x) = x^2 + 3$?

$$\frac{dg}{dx} = 2x$$

What is the derivative of $g(x)$ at $x=5$?

$$\left. \frac{dg}{dx} \right|_{x=5} = 10$$



Review: Derivatives and Gradients

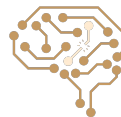
What is the gradient of the function $g(x, y) = x^2y$?

Recall: Gradient is a vector of partial derivatives with respect to each variable

$$\nabla g = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2xy \\ x^2 \end{bmatrix}$$

What is the derivative of $g(x, y)$ at $x=0.5, y=0.5$?

$$\nabla g|_{x=0.5, y=0.5} = \begin{bmatrix} 2(0.5)(0.5) \\ (0.5^2) \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$



Gradient in n Dimensions

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}$$

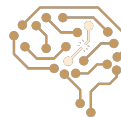


Optimization Procedure: Gradient Descent

```
Init  $w$   
for iter = 1, 2, ...  
     $w \leftarrow w - \alpha \cdot \nabla L(w)$ 
```

L is the loss function

α : learning rate



Calculate Derivatives by Chain Rule

If $f(x) = g(h(x))$

Then $f'(x) = g'(h(x))h'(x)$



Univariate Chain Rule

$$\frac{d}{dt}f(x(t)) = \frac{df}{dx} \frac{dx}{dt}.$$

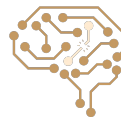
Recall: Loss function for the logistic regression model:

$$z = wx + b$$

$$\hat{y} = \sigma(z)$$

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$$

Let's compute the loss derivatives



Univariate Chain Rule

Computing the loss:

$$z = wx + b$$

$$\hat{y} = \sigma(z)$$

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$$

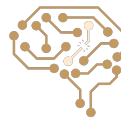
Computing the derivatives:

$$\frac{d\mathcal{L}}{d\hat{y}} = \hat{y} - y$$

$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{d\hat{y}} \sigma'(z)$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz} x$$

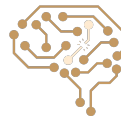
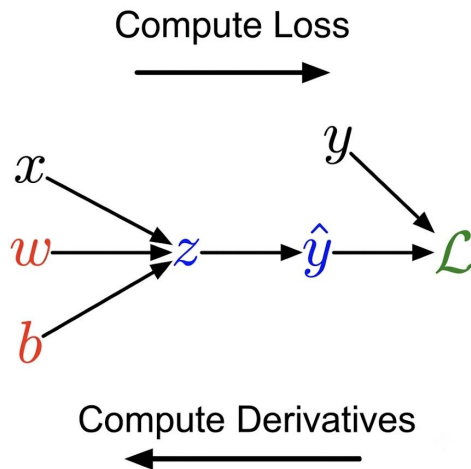
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$



Computation Graph

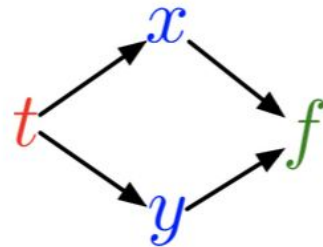
We can diagram out the computations using a computation graph.

The nodes represent all the inputs and computed quantities, and the edges represent which nodes are computed directly as a function of other nodes.



Multivariate Chain Rule

$$\frac{d}{dt} f(x(t), y(t)) = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$



Example:

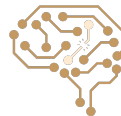
$$f(x, y) = y + e^{xy}$$

$$x(t) = \cos t$$

$$y(t) = t^2$$

Plug in to Chain Rule:

$$\begin{aligned} \frac{df}{dt} &= \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt} \\ &= (ye^{xy}) \cdot (-\sin t) + (1 + xe^{xy}) \cdot 2t \end{aligned}$$



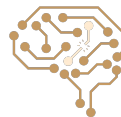
Backpropagation Algorithm

Let $v_1, \dots, v_N$ be a topological ordering of the computation graph (i.e. parents come before children.)

v_N denotes the variable we are trying to compute derivatives of (e.g. loss)

forward pass $\left[\begin{array}{l} \text{For } i = 1, \dots, N \\ \text{Compute } v_i \text{ as a function of } \text{Pa}(v_i) \end{array} \right.$

backward pass $\left[\begin{array}{l} \overline{v_N} = 1 \\ \text{For } i = N - 1, \dots, 1 \\ \overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i} \end{array} \right.$



Backpropagation Example

Gradient of $g(w_1, w_2, w_3) = w_1^4 w_2 + 5w_3$ at $w_1 = 2, w_2 = 3, w_3 = 2$

Think of g as a composition of many functions

- Then, we can use the chain rule to compute the gradient

$$g = b + c$$

$$\frac{\partial g}{\partial b} = 1, \frac{\partial g}{\partial c} = 1$$

$$b = a \times w_2$$

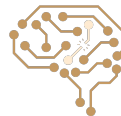
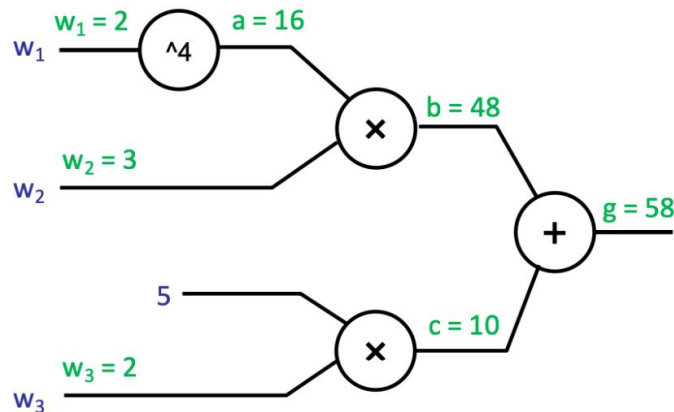
$$\frac{\partial g}{\partial a} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial a} = 1 \cdot w_2 = 3 \quad \frac{\partial g}{\partial w_2} = \frac{\partial g}{\partial b} \frac{\partial b}{\partial w_2} = 1 \cdot a = 16$$

$$a = w_1^4$$

$$\frac{\partial g}{\partial w_1} = \frac{\partial g}{\partial a} \frac{\partial a}{\partial w_1} = 3 \cdot 4w_1^3 = 96$$

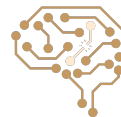
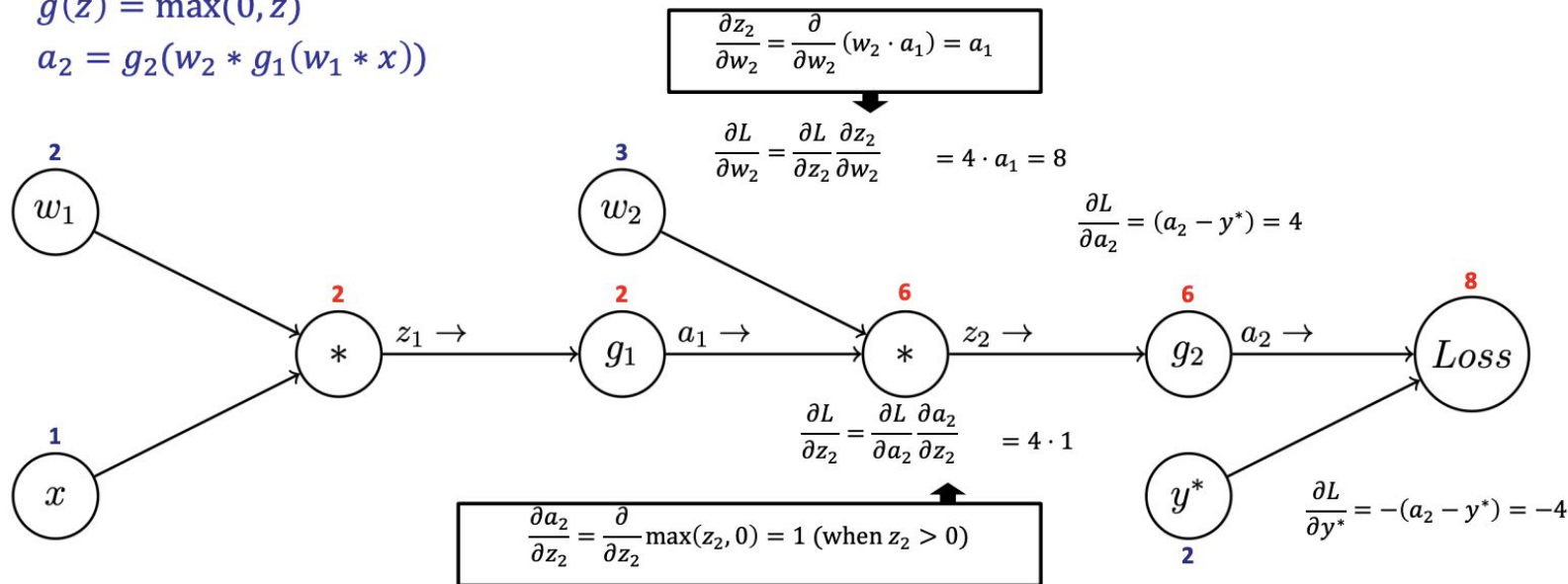
$$c = 5w_3$$

$$\frac{\partial g}{\partial w_3} = \frac{\partial g}{\partial c} \frac{\partial c}{\partial w_3} = 1 \cdot 5 = 5$$



Example: Backpropagation (Do it yourself at home)

- Build a computation graph and apply chain rule: $f(x) = g(h(x)) \quad f'(x) = h'(x) \cdot g'(h(x))$
- Example: neural network with quadratic loss: $L(a_2, y^*) = \frac{1}{2}(a_2 - y^*)^2$ and ReLU activations $g(z) = \max(0, z)$
- $a_2 = g_2(w_2 * g_1(w_1 * x))$



Next Session: Neural Networks Part II

